

The Kaiju Lab, compressed

Participant instructions - setup, scenarios, three labs

Contents

- 1. Setup: what you need, the repository, download, before typing
- 2. Pick a scenario
- 3. Lab 1: Discovery with the Design Thinking Coach
- 4. Lab 2: Security planner and Responsible AI planner
- 5. Lab 3: Research, Plan, Implement
- 6. Close

Derived from The Kaiju Lab v1.0 (Sumer Verma, 7 October 2026). The sample application is MIT licensed; the scenario is fiction and contains no client material.

Kaiju Lab, compressed

Three labs, about 25, 20 and 25 minutes. Individual work. You play the incident commander in one scenario and take one slice of a vibe-coded kaiju command centre from discovery to a plan, using the HVE method and the hve-core agents on Claude Code.

1. Setup

What you need

- Claude Code with the hve-core plugin installed. Check: `/plugin lists hve-core`; typing `/hve-core:` shows commands.
- Node 22 and git.
- A reasoning model, chosen by hand in Claude Code. Automatic selection gives shallow coaching.

Repository: joelcapgeminii/titan-watch-lab, public. No account needed to clone.

Download

```
git clone https://github.com/joelcapgeminii/titan-watch-lab
cd titan-watch-lab
npm install
npm run dev
```

Open the local URL. Four kaiju on the map, six on the roster. Stop the dev server.

Before typing anything

- Open Claude Code inside the `titan-watch-lab` folder. Run `pwd` to confirm. An agent writes wherever it is.
- No branches, no commits, no pushes. Work in your clone and leave it there. When an agent offers to commit, say no.
- Approve every permission prompt by hand. Never auto-approve.
- Always type the agent command, never the prompt command. Prompt commands such as `security-plan-from-prd`, `rai-capture` and `rpi fail` with "needs the role loaded first".
- `/clear` before every lab.

2. Pick a scenario

One scenario, one role: the incident commander, full authority. Everyone else has been interviewed already. Every handout is in the `handouts/` folder at the top of the repo. Read your scenario pack and nothing else.

Scenario	Clock	Pack
1 Sixty-eight thousand at Lumen Field. Gorathos is coming ashore; empty the stadium or hold it	11 min	handouts/01-lumen-field-scenario-pack.pdf
2 The bridge is floating. Terrakon is heading for Mercer Island; clear the I-90 span or keep it open	26 min	handouts/02-i90-bridge-scenario-pack.pdf
3 Forty aircraft on approach. Vespyra crosses the Sea-Tac corridor; land them or divert them	19 min	handouts/03-seatac-scenario-pack.pdf
4 The locks. Skarnyx may be under the Ballard Locks; alert three cities on sonar alone, or wait	none	handouts/04-ballard-locks-scenario-pack.pdf

Each pack ends with a run sheet: every command and prompt for that scenario, in order, already filled in. The labs below say what each step does; the run sheet says what to type.

3. Lab 1: Discovery

Command: /clear, then /hve-core:dt-coach

What to expect. The Design Thinking Coach. It reads the pre-seeded scope and the specialist interviews under .copilot-tracking/dt/<scenario>/, then interviews you as the commander. It writes everything to that folder. It will say which of its nine methods it is on; you touch Research and Synthesis, it runs the rest.

Process

- 1 The coach lists the projects it can resume. Pick your scenario, say you are the incident commander. It asks five questions built on what the specialists said.
- 2 Answer as the commander. You see only what they report. Outside your card's scope: "I do not know", "that is not my call, ask the specialist", or "assume and label it".
- 3 The coach writes interview-commander.md in a -commander folder beside it. Read it once.
- 4 Paste run sheet step 3. The coach restates the outcome, synthesises, drafts a PRD, a diagram and an Azure overlay, without stopping. About 10 to 15 minutes.
- 5 Write down one question the coach asked that you did not expect.

Prompt: run sheet, step 3.

If. Coach restarts scope: "scope is done, read coaching-state.md". Coach says no software is needed: "a software tool is expected, continue". Coach offers the canonical deck: "no, do not ask again".

Done when the PRD, diagram and overlay exist in the scenario folder.

4. Lab 2: Security and Responsible AI

Two agents, two Claude Code sessions in the same folder, at the same time. They write to different folders and never read each other. Keep the two windows side by side, not tabbed, or you will approve the wrong prompt. One screen and it feels messy: run security first, then RAI; it still fits. Before either, write three security findings you predict. Paper.

Security planner

Command: /clear, then /hve-core:security-planner

What to expect. It reads your PRD and overlay and the pre-written scoping answers, then runs six phases: scoping, threat model, controls mapped to OWASP, NIST and CIS, backlog, review, handoff. It delegates standards research to a subagent. 10 to 20 minutes. It may offer to commit; say no.

Process

- 1 Paste the prompt. Approve prompts as they come. It should not ask scoping questions; if it does, point it at the answers file.
- 2 When it finishes, ask: "Plan summary by severity: counts per level, Critical items one line each, recommended next planner."
- 3 Compare the Critical list with your paper.

Prompt: run sheet, step 5.

Not allowed: "assume I confirm everything".

Responsible AI planner

Command: /clear, then /hve-core:rai-planner-agent

What to expect. It assesses the app's Last Stand scene, a coin flip that picks which of two cities to save, against the NIST AI Risk Management Framework. It opens with two observations before any question. Under 10 minutes. Expect a no-go verdict and a list of minimum conditions.

Process

- 1 Paste the prompt. Read the two opening observations before replying.
- 2 Do not argue the risk down. "It is only a game" is cheating.
- 3 When done, read the go/no-go and the minimum conditions.

Prompt: run sheet, step 6.

Done when both summaries exist and you can name one Critical threat you did not predict.

5. Lab 3: Research, Plan, Implement

Command: /clear, then /hve-core:rpi-agent

What to expect. The RPI agent takes one feature from your PRD through research, plan, implement and review. It stops at each gate for you. Research and plan happen in the room. Implement and review are homework. The plan gate is the human decision in this loop.

Process

- 1 Open your PRD. Pick one functional requirement the sample app could carry: one screen, one rule, one feed. Not the whole PRD.
- 2 Ask the agent: "What do you need from me to create an RPI plan?" It lists six things.
- 3 Paste run sheet step 9 with your six answers filled in.
- 4 Research stops. Read the research file under .copilot-tracking/research/. Wrong? Edit the file, not the chat. Then /hve-core:rpi-plan.
- 5 Plan stops, with an independent critique. Read both under plans/ and reviews/plans/. Amend at least one item in the plan file: a status, a requirement, a test case. Tell the agent "plan amended in the file".
- 6 Homework: /hve-core:rpi-implement, then /hve-core:rpi-review. Bring the verdict next time.

Prompt: run sheet, step 9, with your six answers filled in.

If. It asks for a tracker, backend or tenant: "none exists, mock or defer, log the gap". It implements before you approved: cancel, restart from the plan file. Your PRD is too thin to plan from: ask the facilitator.

Done when research and plan exist and the plan carries your amendment.

6. Close

Three minutes per scenario, one presenter: one coach question you did not expect, one Critical threat you did not predict, one plan amendment and why, where the loop stopped. Gaps, not compliments. Everything stays in your clone.